

# Simulating Memory: Devices to Systems

ECE2.414 Advanced Memory Circuits and Systems, Assignment 1

Hrishikesh Gawas

September 12, 2026

**Question.** *Should the 2 MB L2 cache in an accelerator be SRAM or STT-MRAM?* The question is carried through five tools, each working at a different level of abstraction: ngspice for the bitcell, CACTI for the SRAM array, NVSim for the STT-MRAM array, Ramulator 2.0 for the DRAM channel behind it, and gem5 for the effect on a running program. Each part's output becomes the next part's input. Stated accuracies are  $\pm 5\%$  for ngspice,  $\pm 10\%$  for CACTI and Ramulator, and  $\pm 20\%$  for gem5, and these compound multiplicatively, so results are reported as ratios between configurations wherever possible.

Sources: <https://github.com/HrishikeshGawasIIITH/SRAM-vs-STT-MRAM>

## 1 Part A: ngspice, 6T SRAM read margin

### 1.1 Setup

One 6T cell on the 45 nm PTM BSIM4 high-performance card. Pull-ups  $W = 0.15\ \mu\text{m}$ , pull-downs  $W = 0.20\ \mu\text{m}$ , access transistors  $W_A = 0.16\ \mu\text{m}$ , all  $L = 0.045\ \mu\text{m}$ . Stored state  $Q = 0$ ,  $\bar{Q} = 1$ , set with `.ic v(q)=0`; only  $Q$  is forced, since  $v(Q) = 0$  is independent of the supply and the cross-coupled pair raises  $\bar{Q}$  to whatever rail is applied.

Bitlines carry  $C_{BL}$  and are precharged by two  $W = 1.2\ \mu\text{m}$  PMOS devices from  $V_{DD}$ , with a  $W = 0.6\ \mu\text{m}$  equalising PMOS across them. Precharge is active low and releases at  $t = 0.75\ \text{ns}$ . The wordline rises at  $t = 1.0\ \text{ns}$  and  $\Delta V$  is measured at  $t = 2.0\ \text{ns}$ , a 1 ns development window. Wordline and precharge amplitudes scale with  $v(V_{DD})$  so they track the supply. Transient step 5 ps over 4 ns.

Read current is taken from the measured swing as  $\bar{I}_{\text{read}} = C_{BL} \Delta V / \Delta t$ .

**Table 1:** Measured operating points,  $C_{BL} = 180\ \text{fF}$  as given.

| Configuration                                                | $\Delta V$ @2 ns<br>(mV) | $\bar{I}_{\text{read}}$<br>( $\mu\text{A}$ ) | $\max v(Q)$<br>(mV) | as % $V_{DD}$ | $v(Q)$ @4 ns<br>(mV) |
|--------------------------------------------------------------|--------------------------|----------------------------------------------|---------------------|---------------|----------------------|
| Task 1 — nominal, $W_A=0.16\ \mu\text{m}$ , 1.1 V, 27 °C     | 727                      | 131                                          | 206                 | 19 %          | 2.9                  |
| Task 2 — wide access, $W_A=0.24\ \mu\text{m}$ , 1.1 V, 27 °C | 851                      | 153                                          | 272                 | 25 %          | 0.8                  |
| Task 4 — nominal, 1.1 V, 85 °C                               | 579                      | 104                                          | 221                 | 20 %          | 15.2                 |
| Task 4 — nominal, 0.7 V, 85 °C                               | 192                      | 35                                           | 104                 | 15 %          | 60.0                 |

### 1.2 Task 1: $\Delta V$ at 2 ns

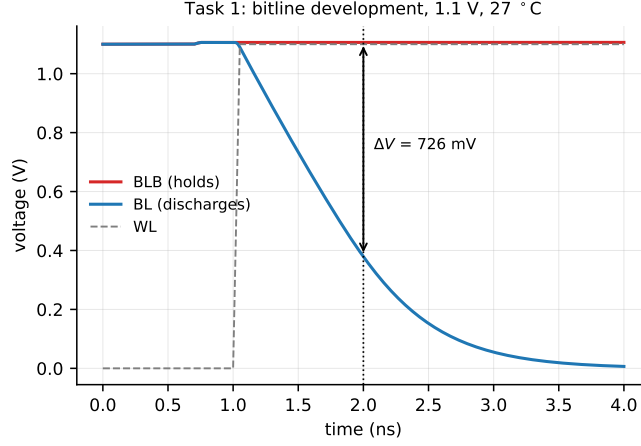
At 1.1 V and 27 °C,  $\Delta V = 727\ \text{mV}$  against the 69 mV reference from Lecture 1. The two are not the same quantity.

69 mV is a DRAM sense margin. DRAM reads by charge sharing, a storage capacitor  $C_s$  connected onto a bitline held at  $V_{DD}/2$ :

$$\Delta V_{\text{DRAM}} = \frac{V_{DD}}{2} \cdot \frac{C_s}{C_s + C_{BL}}.$$

On a 180 fF bitline at 1.1 V, 69 mV corresponds to  $C_s = 25.8\ \text{fF}$ , an ordinary DRAM storage capacitor. The signal has a hard ceiling there, because the cell can only give the bitline the charge it already holds.

An SRAM read has no such ceiling. The access transistor and pull-down form a path to ground and actively discharge the bitline for as long as the wordline is high, so the signal is set by a current and a time rather than by a charge ratio. Fig. 1 shows the result: BL falls to 0.37 V by the sense point and 6 mV by 4 ns, so at 2 ns we are measuring a nearly complete discharge.



**Figure 1:** Bitline development at 1.1 V, 27 °C,  $C_{BL} = 180$  fF.

The swing implies 131  $\mu$ A through a 0.16  $\mu$ m device, about 0.8 mA/ $\mu$ m, normal for a 45 nm high-performance NMOS at 1.1 V. The same netlist on the low-power card gives 248 mV,  $2.9\times$  smaller, so the device family matters more than anything else in the cell.

**Bitline load.**  $C_{BL} = 180$  fF is small for a 2 MB L2. Part B’s winning subarray is 336  $\mu$ m tall, which at a 0.657  $\mu$ m cell height is 512 cells per bitline. Wire alone contributes 89 fF and the drain junctions of 512 unselected cells add to that, putting a real column near 345 fF. Since  $\Delta V$  is  $Q/C_{BL}$  for a fixed current and window, this scales the whole result.

**Table 2:** Same cell and timing, varying the column height.

| Bitline load                          | $\Delta V$ at 2 ns (mV) |             |             |             | margin at worst corner |
|---------------------------------------|-------------------------|-------------|-------------|-------------|------------------------|
|                                       | 1.1 V/27 °C             | 1.1 V/85 °C | 0.7 V/27 °C | 0.7 V/85 °C |                        |
| 180 fF (as given, $\approx 32$ cells) | 727                     | 579         | 256         | 192         | $7.7\times$            |
| 260 fF                                | 525                     | 413         | 180         | 135         | $5.4\times$            |
| 345 fF (CACTI column, 512 cells)      | 402                     | 315         | 137         | 103         | $4.1\times$            |
| 700 fF                                | 203                     | 157         | 70          | 58          | $2.3\times$            |
| 1.8 pF                                | 80                      | 64          | 35          | 34          | $1.4\times$            |

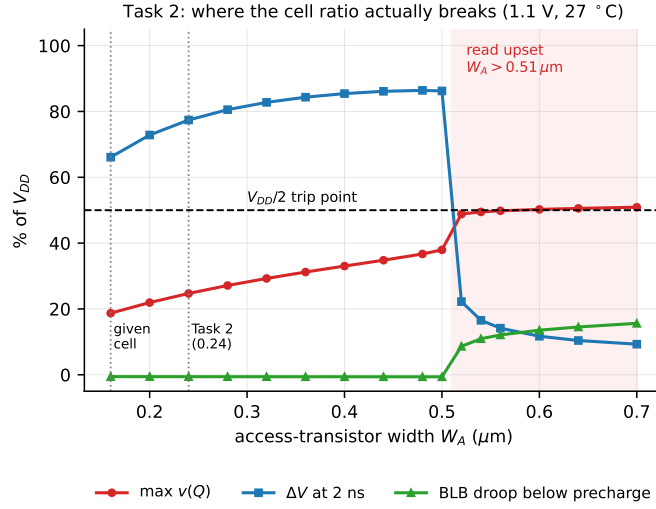
At 345 fF the worst-corner margin is 103 mV,  $4.1\times$  the 25 mV offset instead of  $7.7\times$ . The read still passes with about half the headroom the given netlist suggests. Only at 1.8 pF, which needs roughly 3.3 fF per cell and is too high for 45 nm, does it approach the offset at  $1.4\times$ .

### 1.3 Task 2: storage-node disturb

$Q$  is lifted to 206 mV, 19% of  $V_{DD}$ , because the precharged bitline injects current into the “0” node faster than the pull-down removes it. At  $W_A = 0.24$   $\mu$ m the disturb grows 32% while  $\Delta V$  improves only 17%. The access transistor sets both the read current and the disturb, so widening it helps one and hurts the other.

Neither width flips the cell. The peak stays about a factor of two below  $V_{DD}/2$  and  $Q$  returns to 2.9 mV by 4 ns. Sweeping  $W_A$  (Fig. 2) puts the upset boundary at  $W_A \approx 0.51$   $\mu$ m, a cell ratio of 0.39 against 1.25 as given, so there is roughly  $3.2\times$  margin in access width.

The failure is a cliff rather than a slope because the access transistor works as a source follower during a read. Its gate sits at  $V_{DD}$  while its source is  $Q$ , so as  $Q$  rises the overdrive  $V_{DD} - V_Q - V_{th}$  collapses and the device throttles itself, while the pull-down keeps a full  $V_{DD}$  on its gate. That

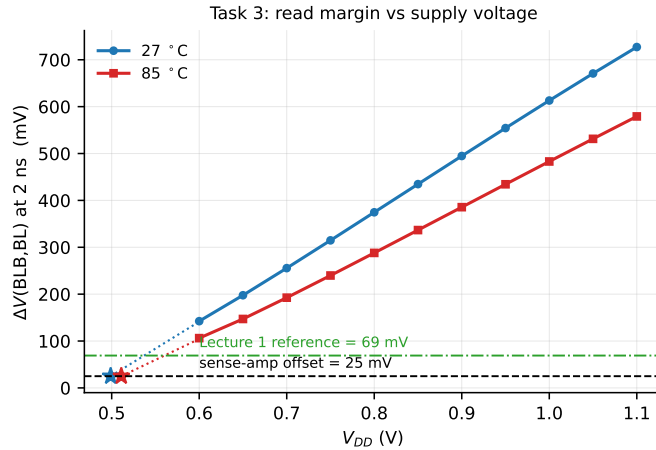


**Figure 2:** Access-width sweep at 1.1 V, 27 °C. Upset is taken from BLB drooping below the precharge rail, which needs  $\bar{Q}$  to have been pulled down.

feedback holds  $Q$  down across a wide range of widths, then stops working once  $Q$  can reach the inverter trip point.

#### 1.4 Tasks 3 and 4: supply and temperature

$V_{DD}$  is stepped 1.1 V to 0.6 V in 50 mV steps at both temperatures (Fig. 3).  $\Delta V$  falls almost linearly, which means the read is discharge limited rather than current limited over this range: BL is pulled nearly to ground at every supply, so  $\Delta V$  follows the precharge level rather than the strongly non-linear  $I_{on}(V_{DD})$ .



**Figure 3:** Read margin against supply. Dotted lines extrapolate the four lowest points to the sense-amp offset.

At 180 fF the margin never reaches the 25 mV offset in this range; the minimum is 106 mV at 0.6 V and 85 °C. Extrapolation puts a crossing near 0.50 V and 0.51 V, but both lie where  $V_{DD}$  approaches  $V_{th} + |V_{th,p}|$  and the linear trend must break.

Read disturb moves the other way, from 19 % of  $V_{DD}$  at 1.1 V to 12 % at 0.6 V, because both devices weaken together while the window is fixed and the source-follower shutoff bites harder when  $V_{th}$  is a larger fraction of the rail. The two failure mechanisms therefore run in opposite directions: at 1.1 V the cell is easy to read and easier to disturb, at 0.6 V it is stable but the signal is small. Both corners have to be checked.

Heating to 85 °C at 1.1 V costs 148 mV, or -20 %. At 0.7 V and 85 °C,  $\Delta V = 192$  mV with  $Q$  disturbed to 15 % of  $V_{DD}$ .

$\Delta V$  moves far more than the offset does. A latch-type sense amplifier’s input-referred offset is dominated by threshold mismatch in its input pair,  $\sigma V_{th} \propto 1/\sqrt{WL}$ , fixed by dopant statistics at manufacture rather than by temperature, so 25 mV is effectively constant here. The margin therefore closes from one side only, and the ratio drops from about  $29\times$  to  $23\times$ . On the  $\Delta V$  side the mechanism is mobility degradation, roughly  $\mu \propto T^{-1.5}$ , weakening the access transistor; this outweighs the opposing fall in  $V_{th}$  at these overdrives. The same mobility loss raises  $Q$  by about 10 %, so read stability degrades at the same time.

**Answer.** The cell reads correctly at 0.7 V and 85 °C, with  $7.7\times$  margin at the given 180 fF and  $4.1\times$  on a column loaded as Part B’s geometry implies. Read stability is the tighter constraint either way, since Task 2 left only  $3.2\times$  in access width against more than  $4\times$  in sense margin. Every number is one simulation at nominal process, so the  $\sigma V_{th}$  spread across the  $2^{24}$  cells of a 2 MB array is not represented.

**Into Part B:** read current  $\approx 131 \mu\text{A}$  per cell.

## 2 Part B: CACTI, sizing the 2 MB SRAM L2

### 2.1 Setup

CACTI 7 as specified: 2 MB, 64 B blocks, 8-way, one read/write port, 4 UCA banks, 45 nm, 350 K, `itrs-hp` for cells and periphery in both the data and tag arrays, ED<sup>2</sup> objective with weights 0:0:0:100:0. Everything else left at CACTI’s defaults. Leakage is reported per bank, so the whole-cache figure is four times that.

**Table 3:** Baseline. This is the SRAM reference for Parts C to E.

| Metric                                                                       | 2 MB SRAM L2, 45 nm |
|------------------------------------------------------------------------------|---------------------|
| Access time (ns)                                                             | 2.90                |
| Cycle time (ns)                                                              | 2.65                |
| Dynamic read energy (pJ/access)                                              | 792.9               |
| Dynamic write energy (pJ/access)                                             | 851.2               |
| Leakage, whole cache (mW)                                                    | 2251                |
| Total area (mm <sup>2</sup> )                                                | 10.66               |
| Area efficiency (%)                                                          | 54.3                |
| Winning organisation (N <sub>dwl</sub> /N <sub>dbl</sub> /N <sub>spd</sub> ) | 4 / 2 / 1           |
| Tag organisation (N <sub>twl</sub> /N <sub>tbl</sub> /N <sub>tspd</sub> )    | 2 / 2 / 2           |

### 2.2 Task 1: baseline

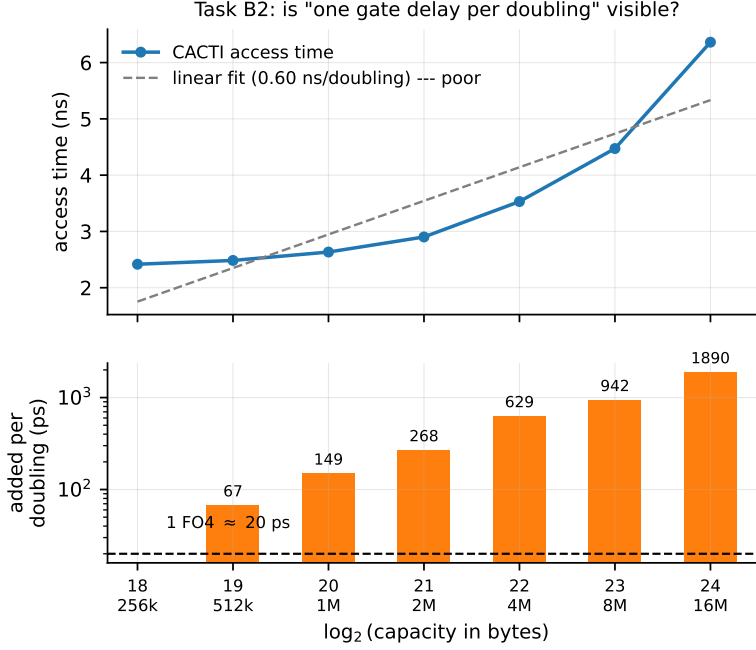
Access time 2.90 ns, read energy 793 pJ, whole-cache leakage 2251 mW, and 10.66 mm<sup>2</sup> of data plus tag array at 54.3 % efficiency. CACTI’s bounding box is  $2.213 \times 5.184$  mm, so 11.47 mm<sup>2</sup> of occupied silicon; that larger figure is what Parts C and E use for the iso-area comparison. The winning organisation is  $N_{dwl}/N_{dbl}/N_{spd} = 4/2/1$ . At 2 GHz the access is 6 cycles, which is the L2 hit latency handed to Part E.

The delay breakdown matters more than the total. Of the 2.90 ns, 47 % is H-tree wire delay in and out of the bank (1.36 ns), 0.54 ns is decoder and wordline, and only 14 % (0.407 ns) is the bitline. Sense amplification is negligible. The access is dominated by interconnect, not by the array, and that is why swapping the bitcell in Part C changes read latency so little.

### 2.3 Task 2: capacity sweep

Capacity swept 256 kB to 16 MB with everything else fixed (Fig. 4).

The “one gate delay per doubling” rule is not visible. It predicts a constant increment of about one FO4 delay, roughly 20 ps at 45 nm. Measured, the increment keeps growing: 67 ps from 256 kB to



**Figure 4:** Access time against capacity, and delay added per doubling. A constant one-gate-delay increment would appear as flat bars.

512 kB, rising to 1.89 ns from 8 MB to 16 MB. The mean cost per doubling is 597 ps, around 30 gate delays, and the curve is convex on a  $\log_2$  axis where the rule predicts a straight line.

The rule describes a decoder-depth limited array, where each doubling adds one level of decode logic and so one gate delay. That held at the nodes where it was measured. At 45 nm a multi-megabyte array is interconnect limited instead: the H-tree is the largest single term at every capacity, wire delay per unit length gets worse with scaling rather than better, and doubling the capacity lengthens the array by  $\sqrt{2}$  in each dimension while RC delay grows with the square of length. The wire terms compound while the logic-depth term stays logarithmic.

## 2.4 Task 3: the objective

The same cache re-run with only the objective changed: pure delay (100:0:0:0:0), pure area (0:0:0:0:100), and ED<sup>2</sup>P.

**Table 4:** Same 2 MB specification, three objectives.

| Objective                | Ndwl/Ndbl/Nspd | Access (ns) | Read (pJ) | Area (mm <sup>2</sup> ) |
|--------------------------|----------------|-------------|-----------|-------------------------|
| Pure delay (100:0:0:0:0) | 4 / 2 / 1      | 2.89        | 898       | 10.21                   |
| Pure area (0:0:0:0:100)  | 2 / 2 / 1      | 3.36        | 797       | 8.65                    |
| ED <sup>2</sup> P        | 4 / 2 / 1      | 2.90        | 793       | 10.66                   |

Delay and ED<sup>2</sup>P both pick 4/2/1. Pure area picks  $N_{dwl} = 2$ , trading 1.2× the access time for 15% less area.

$N_{dwl}$  is the number of vertical cuts through the array, so it sets how many segments each wordline is split into. Doubling it halves the wordline length a row driver has to charge and roughly quarters that RC delay, but every cut also replicates the row decoder, the wordline drivers and a sense-amplifier strip, none of which stores any bits. The delay objective accepts that overhead because it scores only time; the area objective refuses it. There is no optimal cache, only the optimum of the objective written down, and quoting an access time for a 2 MB L2 without naming the objective builds in an error bar the size of the spread.

## 2.5 Task 4: bitline delay hand check

CACTI reports 0.407 ns for the Task 1 winner. The Lecture 4 estimate is  $t = 0.38rcL^2$ . With the winning subarray's  $L = 336\text{ }\mu\text{m}$ , minimum-pitch local wire at  $r = 6.0\text{ }\Omega/\mu\text{m}$  (copper,  $\rho = 2.2 \times 10^{-8}\text{ }\Omega\text{m}$ ,  $45\text{ nm} \times 81\text{ nm}$ ) and  $c = 0.40\text{ fF}/\mu\text{m}$ , the estimate is 0.104 ns, so CACTI's figure is  $3.9\times$  larger.

Two effects account for the gap and both make the hand figure optimistic. The formula counts only wire capacitance, but 512 unselected cells hang off each bitline and their drain junctions at roughly 0.5 fF each add about 256 fF to the 89 fF of wire, nearly four times the wire alone. More importantly,  $0.38rcL^2$  is the step response of an RC line driven by an ideal source that can supply whatever current the line demands. A real bitline is discharged by one access transistor sinking the 131  $\mu\text{A}$  measured in Part A, so the event is current limited and the delay is  $C_{BL}\Delta V/I_{cell}$ , a ramp rather than an RC settling. The lumped formula is the right tool for a wire, and a bitline is a wire plus 512 loads plus a current source.

## 3 Part C: NVSim, the same 2 MB as STT-MRAM

### 3.1 Setup

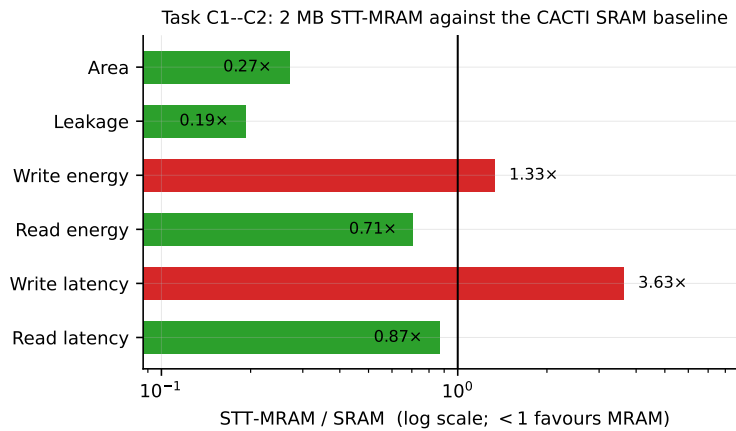
NVSim at 2 MB, 8-way, 512-bit word, 45 nm, 350 K, HP device roadmap, H-tree routing, sequential access mode, read-EDP objective. Only the bitcell differs from Part B.

The cell is as specified: MRAM,  $54\text{ F}^2$ , aspect ratio 2.0,  $R_{on}/R_{off} = 3/6\text{ k}\Omega$ , current read at 40  $\mu\text{A}$ , current set and reset at 200  $\mu\text{A}$  for 10 ns, CMOS access device 6 F wide. Four fields NVSim requires but the given listing omits (`ReadVoltage`, `MinSenseVoltage`, `SetEnergy`, `ResetEnergy`) are taken from NVSim's shipped STT-MRAM sample, with `MinSenseVoltage` set to 25 mV to match Part A.

**Table 5:** 2 MB L2 at 45 nm, SRAM against STT-MRAM. SRAM leakage is the whole-cache figure.

| Metric        | SRAM (CACTI) | STT-MRAM (NVSim) | Ratio         |
|---------------|--------------|------------------|---------------|
| Read latency  | 2.90         | 2.53             | 0.87 $\times$ |
| Write latency | 2.90         | 10.53            | 3.63 $\times$ |
| Read energy   | 792.88       | 562.00           | 0.71 $\times$ |
| Write energy  | 851.22       | 1133.00          | 1.33 $\times$ |
| Leakage       | 2250.53      | 433.93           | 0.19 $\times$ |
| Area          | 10.66        | 2.89             | 0.27 $\times$ |

### 3.2 Tasks 1 and 2: against the SRAM baseline



**Figure 5:** STT-MRAM relative to SRAM, log scale. Below 1 favours MRAM.

**Area and leakage improve sharply.** Area falls  $3.7\times$  and leakage  $5.2\times$ . Both come from the bitcell. A 6T SRAM cell is six transistors, roughly  $140\text{ F}^2$ , and holds its value with a cross-coupled pair permanently tied to the rail, which draws current continuously. An MTJ plus one access

transistor is  $54F^2$  and holds its value in the magnetisation of the free layer, which draws none. The 434 mW that remains is all periphery, so the leakage ratio would improve further at larger capacities.

**Write latency and write energy degrade.** Writes take  $3.6\times$  longer and cost  $1.3\times$  more. This is switching physics rather than periphery. Writing SRAM means overpowering a latch, about a gate delay. Writing an MTJ means passing enough current for long enough to reverse the free layer by spin-transfer torque, a thermally activated process whose switching probability depends on the overdrive above the critical current and on how long it flows. The cell asks for 200  $\mu\text{A}$  for 10 ns, and that pulse is nearly all of the 10.53 ns write latency.

**Read latency and energy barely move**,  $0.87\times$  each. Part B explains it: 47 % of the SRAM access is H-tree wire delay, which does not care what the bitcell is, so changing the cell changes only the array-internal fraction. This comparison also crosses two tools with different peripheral assumptions, so the supportable claim is that read latency is comparable, not that MRAM is faster.

### 3.3 Task 3: TMR ratio

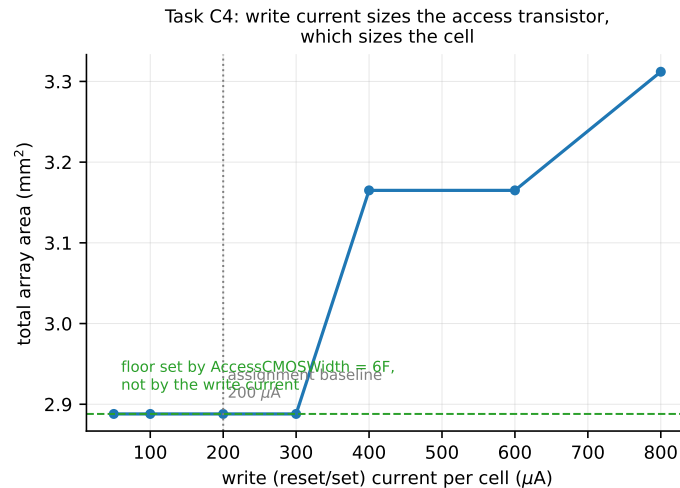
Going from 2:1 to 3:1 ( $R_{on} = 4\text{ k}\Omega$ ,  $R_{off} = 12\text{ k}\Omega$ ) moves read latency 2.53 ns to 2.54 ns, read energy 562 pJ to 561 pJ, and area  $2.89\text{ mm}^2$  to  $2.88\text{ mm}^2$ . Sweeping the ratio from 1.5:1 to 6:1 changes read latency by 0.5 % in total. Nothing moves.

The reason is a threshold both designs already clear. With current-mode sensing the available signal is  $\Delta V_{\text{sense}} = I_{\text{read}}(R_{\text{off}} - R_{\text{on}})$ , so  $40\text{ }\mu\text{A} \times 3\text{ k}\Omega = 120\text{ mV}$  at 2:1 and  $40\text{ }\mu\text{A} \times 8\text{ k}\Omega = 320\text{ mV}$  at 3:1, against a 25 mV requirement. The amplifier resolves in the same time either way, leaving the read path dominated by decode, H-tree and sense-amp periphery, which the cell resistance does not touch.

What TMR actually buys is sense margin, the separation between the “0” and “1” distributions relative to their spread from MTJ variation and reference-cell offset. That is read reliability, so bit error rate and hence yield and ECC overhead, rather than speed or energy. Extra TMR only helps while TMR is the binding constraint, at small read current, high temperature or with a noisier amplifier. Above the threshold it is margin already paid for that cannot be spent elsewhere, and a deterministic array model has no variation distribution with which to price it.

### 3.4 Task 4: write current and cell area

Halving `ResetCurrent` from 200 to 100  $\mu\text{A}$  changes area by 0 %. Sweeping the current over a wider range shows why (Fig. 6): area is flat from 50 to 300  $\mu\text{A}$ , steps up at 400  $\mu\text{A}$ , and reaches  $3.31\text{ mm}^2$  at 800  $\mu\text{A}$ . Below the knee only the write drivers change, because the cell area is an input to the tool rather than something it derives, and nothing checks that the declared access transistor can supply the declared current. The coupling has to be closed by hand.



**Figure 6:** Array area against per-cell write current.

**Table 6:** Cell variants. The last row resizes the access transistor and the cell to match the halved write current.

| Cell variant                                         | Read<br>(ns) | Write<br>(ns) | Read<br>(pJ) | Write<br>(pJ) | Leak<br>(mW) | Area<br>(mm <sup>2</sup> ) |
|------------------------------------------------------|--------------|---------------|--------------|---------------|--------------|----------------------------|
| Baseline (3 / 6 k $\Omega$ , 200 $\mu$ A)            | 2.53         | 10.53         | 562          | 1133          | 434          | 2.888                      |
| TMR 3:1 (4 / 12 k $\Omega$ )                         | 2.54         | 10.54         | 561          | 1132          | 434          | 2.884                      |
| Write current halved (100 $\mu$ A)                   | 2.53         | 10.51         | 560          | 1131          | 434          | 2.888                      |
| 100 $\mu$ A with cell resized to 38.4 F <sup>2</sup> | 3.66         | 11.68         | 299          | 945           | 131          | 1.727                      |

The access transistor has to supply the full write current, and an MOS device delivers current in proportion to its width, so  $W \propto I_{write}$ . In a 1T-1MTJ cell the junction sits above the metal stack and takes no silicon of its own, so the cell footprint is the access-transistor footprint.

For this cell: 54 F<sup>2</sup> at aspect ratio 2.0 is 10.392 F  $\times$  5.196 F, and the 6 F device lies along the long dimension, leaving 10.392 – 6 = 4.392 F of contact and spacing. Halving the current halves the width, so 6 F becomes 3 F, the long dimension becomes 7.392 F and the cell falls to 38.4 F<sup>2</sup>, down 28.9 %. Set current halves too, since one transistor drives both directions. Feeding that cell back in gives the last row of Table 6: area 2.89 mm<sup>2</sup> to 1.73 mm<sup>2</sup>, leakage 434 mW to 131 mW. The array shrinks by more than the cell did because the organisation is re-optimised around the smaller cell, and leakage falls further still because subthreshold current through the access device scales with its width.

So the chain is: write current sets the access-transistor width, the width sets the cell area, the cell area sets density, and density sets the capacity available at fixed silicon, which is what Part E shows moves runtime. A device that halves switching current is not mainly offering lower write energy, which is the small prize. It is offering a 28.9 % smaller cell, about 1.4 $\times$  more capacity at iso-area. Conversely a device needing 800  $\mu$ A gives back 15 % of the density it was chosen for. Reducing switching current moves the metric the technology is selected for; raising TMR moves one this class of tool cannot resolve.

## 4 Part D: Ramulator 2.0, the L2 miss stream on DDR4

### 4.1 Setup

DDR4-3200AA, 8 Gb x8, one channel, two ranks, open-row policy, FR-FCFS scheduling, all-bank refresh, RoBaRaCoCh address mapping. Relevant timings are  $t_{RCD} = 22$ ,  $t_{RP} = 22$ ,  $t_{RAS} = 52$ ,  $t_{CL} = 22$  and  $t_{CCD} = 4$  short / 8 long cycles, with  $t_{CK} = 0.625$  ns. The organisation gives  $2 \times 4 \times 4 = 32$  independent banks.

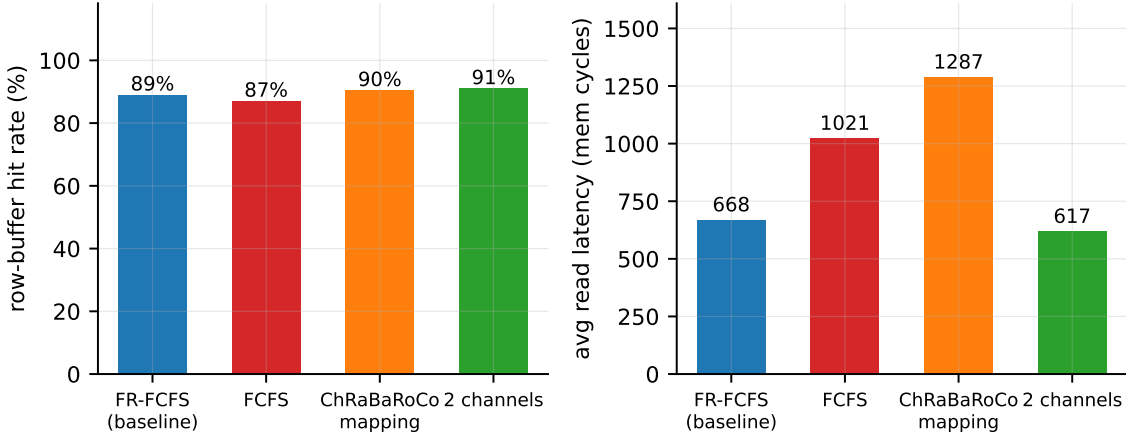
The trace is 230296 reads and 69704 writes taken from the memory-side port of the Part E baseline L2, running `bfs` over the same scale-18 graph. Evictions that carry no data to DRAM are excluded; writebacks are kept.

Requests are replayed one per frontend cycle regardless of whether the previous one completed, so the channel is saturated and absolute latencies are dominated by queueing rather than by DRAM timing. Only ratios between configurations are meaningful. Average read latency is accumulated read latency divided by the memory system’s own read count.

**Table 7:** Four configurations on the same trace. Latencies in memory cycles.

| Configuration                        | Memory<br>cycles | Avg read<br>(cycles) | (ns) | Row-buffer<br>hit rate | ACT<br>commands |
|--------------------------------------|------------------|----------------------|------|------------------------|-----------------|
| FR-FCFS, RoBaRaCoCh, 1 ch (baseline) | 2143874          | 668                  | 417  | 88.9 %                 | 33261           |
| FCFS, RoBaRaCoCh, 1 ch               | 3696183          | 1021                 | 638  | 86.9 %                 | 39215           |
| FR-FCFS, ChRaBaRoCo, 1 ch            | 4517139          | 1287                 | 804  | 90.3 %                 | 29058           |
| FR-FCFS, RoBaRaCoCh, 2 ch            | 978173           | 617                  | 386  | 90.9 %                 | 13631           |

Part D: scheduler, address mapping and channel count



**Figure 7:** Row-buffer hit rate and average read latency across the four configurations.

#### 4.2 Task 1: baseline

FR-FCFS on one channel takes 2143874 cycles, average read latency 668 cycles (417 ns), row-buffer hit rate 88.9 %.

#### 4.3 Task 2: FCFS

Strict first-come-first-served costs  $1.72\times$  in cycles and  $1.53\times$  in read latency, +354 cycles per read. The row-buffer hit rate does not collapse. It falls 88.9 % to 86.9 %, about two points, and activations rise 18 %, 33261 to 39215. Those extra activations do not account for the slowdown. Each row conflict costs  $t_{RP} + t_{RCD} = 44$  cycles, so 5954 more of them is about 261976 cycles against a total slowdown of 1552309 cycles. Row conflicts explain roughly 17 % of it.

The other 83 % is bank-level parallelism. A BFS miss stream over a CSR graph has strong spatial locality, since consecutive misses walk an edge list, so arrival order is already close to row order and most accesses find an open row whatever the scheduler does. Row hits come from the access pattern, not from reordering. What FR-FCFS provides is the freedom to promote a request whose bank is idle over one whose bank is still inside its  $t_{RAS}$  window, keeping the bus busy while a bank recovers. FCFS cannot: the oldest request is by definition next, so the controller idles through the activation shadow even when other banks are free.

#### 4.4 Task 3: row bits below the bank bits

ChRaBaRoCo places the row field below the bank field, so a sequential sweep walks every column of a row and every row of a bank before reaching the next bank. This costs  $2.11\times$  in cycles at 1287 cycles of read latency, the worst of the four and worse than FCFS.

Row-buffer behaviour again does not explain it. The hit rate is higher than baseline, 90.3 % against 88.9 %, with fewer activations (29058 against 33261).

**Table 8:** Bank-level parallelism measured over the trace.

| Address mapping | banks reached<br>(of 32) | mean banks per<br>64-access window | busiest bank's<br>share of traffic |
|-----------------|--------------------------|------------------------------------|------------------------------------|
| RoBaRaCoCh      | 32 / 32                  | 4.85                               | 4.3 %                              |
| ChRaBaRoCo      | 1 / 32                   | 1.00                               | 100.0 %                            |

Extracting the bank index for every access shows what is lost. The baseline mapping reaches all 32 banks and touches 4.85 distinct banks in a typical 64-access window. ChRaBaRoCo reaches 1 bank and 100 % of traffic lands in it. With 10 column bits and 16 row bits below the bank field the bank

index changes only every  $2^{26}$  transactions, far beyond the working set, so the whole trace falls into one bank of 32.

That also explains the higher hit rate. Serialising into one bank backs the queue up much further, which gives FR-FCFS a larger window in which to batch same-row requests. Meanwhile one bank cannot overlap its own  $t_{RP}$  and  $t_{RCD}$  with another bank’s column transfers, and 31 idle banks cannot help however well the one in use is scheduled. The configuration with the best row-buffer hit rate is the slowest, so hit rate alone is not a useful proxy for DRAM performance.

#### 4.5 Task 4: channel count

Two channels improve read latency 8% and more than double throughput at  $2.19\times$ . Extending the sweep gives  $4.41\times$  at four channels and  $8.99\times$  at eight, with latency levelling off around 667 and 620 cycles.

The binding parameter can be identified from the output, and it is not  $t_{RCD}$  or  $t_{RP}$ . At a 88.9% hit rate only about one access in ten activates, and the run issues 0.0155 activations per cycle against a ceiling of 0.4324 set by  $t_{RC} = 74$  across 32 banks, so activation runs at 3.6% of capability. The same argument rules out  $t_{RAS}$ .

What binds is  $t_{CCD}$ . The baseline sustains 0.1399 accesses per cycle per channel, above the same-bank-group floor  $1/t_{CCD\_L} = 0.125$  and around 60% of the cross-group floor  $1/t_{CCD\_S} = 0.250$ , so the data bus is being driven at a rate only reachable by interleaving column commands across bank groups. Each channel added brings its own bus and its own  $t_{CCD}$  budget, which is why throughput scales nearly linearly to four channels. The flattening beyond that is not a DRAM limit: the replay cannot inject fast enough to keep eight buses busy. Improving latency further means attacking queueing, through fewer outstanding misses or more bank groups per channel, rather than adding channels.

The miss traffic does not fit comfortably in one channel. A superlinear  $2.19\times$  from the second only happens if the first was saturated.

### 5 Part E: gem5, the effect on a running program

#### 5.1 Setup

One X86 core at 2 GHz in syscall-emulation mode, 32 kB 8-way L1I and L1D at 2-cycle latency, 8-way L2, DDR4-2400 8x8 memory, 4 GB address space. Kernels are GAPBS `bfs` and `sssp` on a scale-18 Kronecker graph, 262 143 nodes and 3.8 M undirected edges, built beforehand and loaded from file so the simulated region is the traversal rather than the graph generator.

Each kernel runs two trials. Setup and the first trial execute on an atomic CPU with the caches live, and the detailed core takes over at the trial boundary, so what is measured is a warmed steady-state trial. The switch points are the single-trial instruction counts measured separately, 20.7 M for `bfs` and 131.1 M for `sssp`. Statistics are reset at the switch, and every configuration executes the identical second trial.

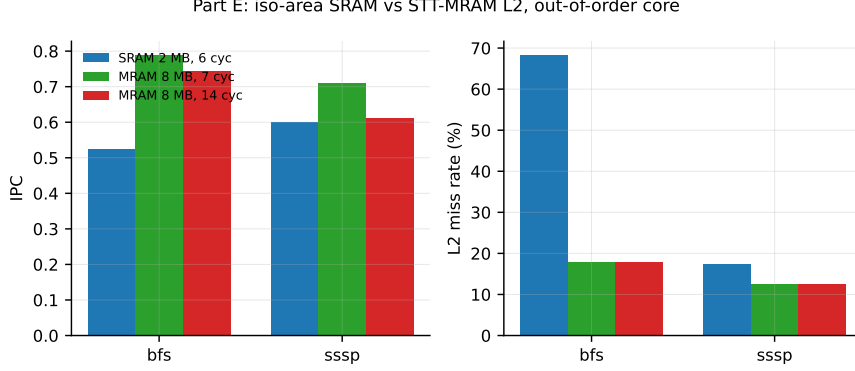
L2 capacities and hit latencies come from Parts B and C:

- **SRAM, 2 MB.** CACTI 2.90 ns, so 6 cycles at 2 GHz, in  $11.47 \text{ mm}^2$ .
- **STT-MRAM, 8 MB.** NVSim 3.21 ns, so 7 cycles, in  $10.32 \text{ mm}^2$ .

8 MB is the iso-area capacity, so the swap is  $4\times$  the capacity in the same silicon. The measured read penalty is one cycle rather than a factor of two, because 47% of the access is H-tree wire delay and does not depend on the cell. Since the comparison turns on that number, a third configuration re-runs the MRAM at 14 cycles as a sensitivity check.

**Table 9:** Out-of-order core, warmed steady-state trial.

| Kernel | L2 configuration  | IPC    | L2 miss rate | simSeconds |
|--------|-------------------|--------|--------------|------------|
| bfs    | SRAM 2 MB, 6 cyc  | 0.5245 | 0.6824       | 0.016183   |
| bfs    | MRAM 8 MB, 7 cyc  | 0.7883 | 0.1798       | 0.010768   |
| bfs    | MRAM 8 MB, 14 cyc | 0.7439 | 0.1800       | 0.011411   |
| sssp   | SRAM 2 MB, 6 cyc  | 0.6011 | 0.1729       | 0.107599   |
| sssp   | MRAM 8 MB, 7 cyc  | 0.7093 | 0.1259       | 0.091195   |
| sssp   | MRAM 8 MB, 14 cyc | 0.6109 | 0.1256       | 0.105882   |

**Figure 8:** IPC and L2 miss rate for the three L2 configurations.

## 5.2 Tasks 1 and 2: which kernel benefits

At the measured latencies the MRAM L2 is faster on both kernels,  $1.503\times$  on **bfs** and  $1.180\times$  on **sssp**. Held at the pessimistic 14 cycles, **bfs** keeps  $1.418\times$  while **sssp** falls to  $1.016\times$ , essentially a wash.

The deciding property is how much of the miss rate the extra capacity removes. **bfs** misses 68.2% at 2 MB and 18.0% at 8 MB,  $3.8\times$  fewer trips to DRAM. At scale 18 it streams a 32 MB edge array once, with no reuse any cache can hold, but it repeatedly touches much smaller metadata: the parent array at about 1 MB plus the frontier and visited bitmaps. That hot set sits just above 2 MB and comfortably inside 8 MB, which is the condition for a capacity trade to pay.

**sssp** starts at 17.3% and only reaches 12.6%, a factor of 1.4. Its delta-stepping inner loop does far more work per byte fetched, so most of its reuse is already captured at 2 MB and little miss traffic is left for the extra capacity to remove, while the added hit latency is charged on the roughly 87% of accesses that already hit.

Capacity pays off on misses and latency taxes hits, so the ratio of the two decides the outcome. A kernel whose reused footprint straddles the two capacities wins clearly; one already resident at the smaller size pays the tax and collects little.

## 5.3 Task 3: in-order core

**Table 10:** In-order core, same two configurations.

| Kernel | L2 configuration | IPC    | L2 miss rate | simSeconds |
|--------|------------------|--------|--------------|------------|
| bfs    | SRAM 2 MB, 6 cyc | 0.1892 | 0.6854       | 0.044869   |
| bfs    | MRAM 8 MB, 7 cyc | 0.2628 | 0.1577       | 0.032304   |
| sssp   | SRAM 2 MB, 6 cyc | 0.1669 | 0.1669       | 0.387665   |
| sssp   | MRAM 8 MB, 7 cyc | 0.1770 | 0.1254       | 0.365425   |

In order, the advantage shrinks on both kernels: **bfs**  $1.503\times$  to  $1.389\times$ , **sssp**  $1.180\times$  to  $1.061\times$ .

Per-miss accounting shows why. On **bfs** each avoided L2 miss is worth 117 cycles in order but only 50 cycles out of order, and on **sssp** 113 against 73. The in-order machine has one miss outstanding at a time, so an avoided miss returns a nearly fully exposed DRAM round trip. The out-of-order

core keeps several in flight through its MSHRs, so they overlap and any single miss sits only about half on the critical path.

Out-of-order execution was therefore already hiding roughly half the cost of every L2 miss, which is the same mechanism that makes the MRAM's extra hit latency cheap there. Removing the reordering exposes those extra cycles as stalls, so the latency half of the trade is paid in full. **bfs** survives because its capacity benefit dominates anyway; **sssp** ends barely ahead. The Task 1 result is partly a property of the core, not only of the memory technology, and should be quoted with the microarchitecture attached.

#### 5.4 Task 4: the argument

**(i) SRAM or STT-MRAM?** STT-MRAM, as an 8 MB iso-area replacement, provided the kernels resemble **bfs** and the core stays out of order.

The case is a chain of measurements. Part A measured 131  $\mu\text{A}$  of read current per 6T cell and confirmed the cell reads correctly at 0.7 V and 85 °C with  $4.1\times$  margin on a realistically loaded column. Part B sized that baseline at 2.90 ns, 793 pJ, 2251 mW and 11.47 mm<sup>2</sup>, and showed 47 % of the access is wire rather than array. Part C fitted 8 MB of STT-MRAM into the same area with  $5.2\times$  less leakage and  $3.7\times$  the density, at  $3.6\times$  the write latency; because the access is wire-dominated the read penalty is one cycle. Part D showed the resulting miss stream needs more than one DDR4 channel. Part E turns it into  $1.503\times$  on **bfs** and  $1.180\times$  on **sssp**, plus a standing leakage saving no IPC figure records.

The cost is one-sided. Writes take 10.53 ns against 2.90 ns, and that penalty comes from the 10 ns switching pulse rather than from the periphery, so it will not be engineered away. For a read-dominated accelerator streaming weights and activations that is acceptable; for a write-intensive one the answer inverts.

With its conditions attached: 8 MB of STT-MRAM is 50.3 % faster than 2 MB of SRAM on **bfs** and 18.0 % on **sssp**, at iso-area, given these device numbers, this cell file, an FR-FCFS controller and an out-of-order core.

**(ii) Three assumptions to attack first.** *The hit-latency ratio, which comes from comparing two different models.* CACTI reports 2.90 ns for the 2 MB SRAM array and NVSim 3.21 ns for the 8 MB MRAM array at matched capacity, node, temperature and roadmap, but the two searched different organisation spaces and CACTI spends 1.36 ns in the H-tree alone. Changing that one input moves **bfs** from  $1.503\times$  to  $1.418\times$  and **sssp** from  $1.180\times$  to  $1.016\times$ . Both arrays should be modelled in one tool before any latency ratio is believed.

*The DRAM timing conclusions, given a saturated replay.* Part D injects one request per cycle regardless of completion, so the inter-arrival timing that actually decides whether traffic fits in a channel is discarded. The 668 cycle average is mostly queueing, which is why two channels nearly halve total cycles while moving latency only 8 %. Those ratios should be re-measured with gem5 driving Ramulator directly.

*The coverage, which is two kernels on one graph.* Every Part E figure is one scale-18 graph, one trial per configuration, no error bars, against an accuracy budget of  $\pm 20\%$  for gem5 and  $\pm 10\%$  each for CACTI and NVSim, compounding multiplicatively. **bfs** at  $1.503\times$  survives that. **sssp** at  $1.180\times$ , and  $1.016\times$  under the pessimistic latency, does not, so the sign of the **sssp** conclusion is not resolved by this experiment. The density advantage the whole argument rests on is also one line of a cell file: Part C showed halving the switching current shrinks the cell 28.9 % and the array to 1.73 mm<sup>2</sup>, which buys more capacity at fixed area than any peripheral optimisation in Part B.